

Ground, then Act: Harness-Level Reliability for an Agent on CAR-bench

shiina

Independent researcher
jaren1418@gmail.com

Abstract

CAR-bench scores an in-car voice agent on *reliability*, not peak capability. A task passes only if it is solved in all three independent trials (Pass³); under the per-trial reward, a single failed dimension zeroes the trial. Frontier models exhibit a “Completion > Compliance” pattern—fabricating success when a capability is absent and guessing values when they are underspecified—even though the reward already credits honest refusal. We submit a ReAct agent whose reliability comes from three model-agnostic mechanisms: (1) **capability honesty**—read state, then admit a missing capability instead of blind-calling; (2) **preference-grounded action**—a bounded LLM judge withholds guessed values on preference-backed writes and forces resolution from stored user preferences; (3) **commit-then-act gating**—deterministic policy and schema checks run on proposed calls before dispatch. All gates are pure code keyed by session state; the LLM is used only for narrow classification. On the public train set the agent improves Pass³ from 47.7% to 60.4% (+12.7pp; hallucination +25pp, disambiguation +13pp, base parity), cheaply and quickly—roughly \$0.001 per trial at 1.06× the baseline’s latency (a 94.7% prompt-cache hit)—on a non-frontier model. The gain holds whether reasoning is on or off; we make no claim about frontier-scale substitution.

1 Introduction

CAR-bench [2026] is a 254-task, 58-tool benchmark of in-car voice agents under real-world uncertainty, scored by Pass³ (a task counts only if solved in all three trials). Its three task families induce distinct failure modes: multi-turn completion under mandated pre-checks (Base), admitting a removed tool, parameter, or result field (Hallucination), and resolving an underspecified value (Disambiguation). Because the per-trial reward is a conjunction of fine-grained dimensions, a single slip—a skipped probe, an execution error, a policy breach—zeroes the trial.

The organizers document a striking default in frontier systems: models prioritize *claiming success over admit-*

ting incapability and guessing over clarifying—a “Completion > Compliance” pattern. We treat this as a behavioral defect addressable at the *harness* level—the runtime scaffolding around the model, not the model weights—rather than a deficit requiring a larger model. The key observation is that the reward model already rewards the correct behavior on hallucination tasks: a trajectory that reads state, discovers the gap, and tells the user honestly scores pass (no execution error, and the simulated user registers an admission). The default scaffold simply never steers the agent toward that trajectory.

Contributions. (i) A single-pass ReAct agent whose reliability comes from an owned prompt and three runtime-detection gates—capability honesty (admit a missing tool, never blind-call), a preference-grounded judge, and deterministic commit-then-act policy gating—all compliant by construction. (ii) This lifts Pass³ from 47.7% to 60.4% on the public train set (+12.7pp; hallucination +25pp, disambiguation +13pp, base parity) at 1.06× latency, ~\$0.001/trial, and a 94.7% prompt-cache hit, on a non-frontier model. (iii) The gain is orthogonal to model reasoning—it stacks with a thinking mode to 68.3% train / 62.0% test—so reliability is bought at the harness, not by scaling the model.

2 Background

A2A session model. The organizer-run evaluator owns the loop and executes all tool calls; one `execute(context)` call is one assistant step. Track 1 imposes no sequential-LLM-call cap, so the agent may make several internal calls per step and persist state across steps—which is what licenses an internal check node.

Reward model. The per-trial reward starts at 1 and subtracts 1 per failing dimension {final state, intermediate state, required tool subset, tool execution, conversation close, policy}, floored at 0. Hallucination tasks score *only* tool execution and conversation-close, so the outcome hinges on whether the simulated user hears an admission of the removed capability—the lever our first mechanism exploits.

3 Method

The agent keeps the baseline ReAct loop [Yao *et al.*, 2022]—one `execute(context)` call per assistant step—and adds

two harness-level, model-agnostic *components* (Figure 1): an *owned prompt*—a curated system prompt we control, replacing the verbatim evaluator prompt—that flips the model’s default stance from “force” to “admit,” and a *dispatch-time check node* whose three gates—schema, preference, and policy—are the three mechanisms summarized above (capability honesty, preference grounding, and commit-then-act gating). Both components surface problems at runtime only—a schema mismatch, an unknown field, a guessed value, or a policy breach. The check node is a guardrail, but unlike a general-purpose guardrail that moderates content independently of the task, it is coupled to the benchmark’s policy wiki and aligned to its reward—a withheld call is one the reward would have penalized.

Owned prompt: admit, do not force. The model’s default—force or fabricate through a missing capability rather than admit it—is exactly what the reward penalizes. On turn one we install a preamble that reverses it, whose load-bearing “Missing Capabilities: Admit, Do Not Force” section tells the agent that if a tool, parameter, or result field is absent it must say so to the user and stop, never fabricating a value or improvising a workaround. The preamble also fixes output rules (the reply drives a text-to-speech model, so no markdown or formatted lists) and reactivity (a state-changing action follows only an explicit call-for-action, a confirmation, or a selection). The rule-bearing policy sections pass through verbatim; our contribution is the framing, not the policy text. The framing is what makes the check node’s honesty mechanism bite: steered to admit, the model stops fighting a gate that withholds a missing capability.

Check node: gate loop. Dispatching a non-compliant call costs the whole trial, so a check node intercepts every proposed batch before it reaches the user agent and runs it through three gates—schema, preference, policy. A batch that clears all three is emitted unchanged. A batch that hits any gate is *withheld*: it is kept in the agent’s internal context together with a synthetic tool-result that tells the LLM what was wrong, and the LLM re-proposes. Only the final gated action reaches the user agent; withheld proposals are never dispatched (they are not hidden emitted calls), and the gates’ state extraction operates on a filtered view that sees only what actually executed. The loop runs at most three attempts; if it is exhausted, the agent makes one tool-less call that tells the user honestly what it cannot do—falling back to a code-authored refusal if that call fails—so the code never forces a tool. Honest refusal is a terminal action, not a failure mode: on hallucination tasks it is exactly the trajectory the reward scores as pass.

Schema gate. The schema gate catches the root cause of hallucination failures before dispatch: a call to a parameter the tool does not actually expose. The system text or a policy clause often describes such a parameter, the model trusts it and calls with it, and the gate withholds the call—turning a certain runtime execution error into an in-step correction.

Preference-grounded judge. When the agent sets a value the user did not state explicitly, the policy requires reading the user’s stored preference for it rather than guessing. A bounded LLM judge inspects each such proposed

value, returning *grounded* when it traces to an explicit user statement, a policy rule, the conversation context, or an already-read stored preference, and *guess* when none of these traces it. A guess is withheld with a hint to resolve the value through the disambiguation priority ladder (read `get_user_preferences`); once preferences have been read the judge short-circuits and is not called again. It is fail-safe: on any failure or ambiguity it returns *grounded*, so it can never hard-block a legitimate action.

Policy gate. The policy gate ensures each proposed action complies with the vehicle’s operating policy before dispatch—turning the air conditioning on must also close any window open more than 20% and raise the fan; fog lights must bring the low beams on and the high beams off; a high beam must not be lit while the fog lights are on. Each rule is a deterministic code check. The cascade rules (Appendix A) combine two requirements: a procedural one—the relevant state must have been read in a prior step, not in the same batch, since checking and acting together would defeat a check-before-acting rule—and a final-state one—the predicted post-batch state must satisfy the rule. A value the agent has never read is never guessed: a rule either skips, because it cannot judge, or blocks until the agent reads first. Detection is all code; only the confirmation rules (Appendix A) leave the question of whether the user actually agreed to a second, separate LLM judge.

Result-side disclosure. When a returned result contains a field the system could not determine (valued `unknown`), the gate forces the agent to name it proactively in its next reply; left to itself the model stays silent, and a missing value goes undisclosed. A code check detects the `unknown` value and injects a notice, so the gap is surfaced even on a generic request.

LLM call sites. The LLM only proposes actions and makes two narrow semantic judgments, both fail-safe. The first is the preference-grounded judge above. The second is a separate *confirmation judge*: three confirmation rules (LLM-POL:004/007/008) mark a proposal as requiring confirmation and block it until the user replies; on the next turn the judge decides whether the reply actually confirms, fail-safely promoting on any error. Keeping LLM judgment this narrow is deliberate: a broader text-level judge over-fires on legitimate actions, which we found regressed disambiguation at full scale. All cross-step state is keyed by `context_id` with no hidden state.

4 Results

Setup. Two public splits—**train** (50 Base, 48 Hallucination, 31 Disambiguation) and **test** (50/50/25)—three trials each (387 / 375 runs). One model throughout: DeepSeek V4-Flash [Xu *et al.*, 2026], 50-step cap. We report two thinking settings on that same model: *thinking off* (temperature 0, the default) and *thinking on, high* (reasoning effort). Overall is the macro-average of the three task-type Pass³ rates. The frozen baseline is the official starter agent, pinned unchanged; all gated rows are our agent.

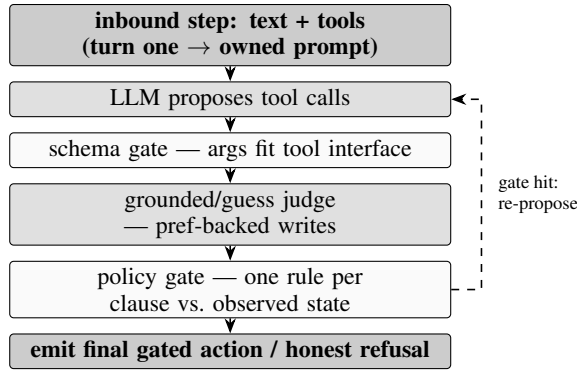


Figure 1: The check node. Shaded nodes are LLM calls (propose, plus the narrow grounded/guess judge); unshaded nodes are pure deterministic code. The dashed edge is the withhold-and-re-propose loop: a rejected batch is retained in internal context with a synthetic tool-result and re-proposed (at most three attempts); only the final gated action is emitted. All state is keyed by `context_id` (not shown); there is no hidden state.

Table 1: Pass³ (macro over task types) on both public splits. Same model (V4-Flash) throughout; rows differ by architecture and thinking. Cost and latency are agent-only, per trial (a Pass³ task runs 3 trials).

Config	Train	Test	\$/trial	s/trial
Baseline, think off	47.7	43.3	0.0010	12.4
Ours, think off	60.4	54.7	0.0011	13.2
Ours, think high	68.3	62.0	0.0042	27.5

Main result. Our agent raises Pass³ by +12.7pp on train and +11.4pp on test over the frozen baseline at the same thinking setting, and the gain clears the split: test sits within 6.3pp of train at every row, so the mechanisms generalize rather than overfit. The gains land where the harness acts—hallucination +25pp and disambiguation +13pp on train (Tables 2, 3)—with base at parity. The test hallucination set is intrinsically harder—the baseline also drops ~20pp train→test (42→22)—so the fair comparison is the relative gain, which holds up (+25pp train, +28pp test). Per-task flips across trials are LLM-sampling and user-simulator noise, so we read the macro over the full split rather than point estimates. Agent-only latency is 1.06× baseline (13.2 vs 12.4 s/trial).

Ablation (mechanism attribution, train). The build-up is cumulative on train. The owned prompt and the honesty gates alone—those that detect missing parameters and unknown result fields—reach 54.6% (+6.9pp), *entirely* from hallucination (+12 tasks): blind-call and text-commit failures become honest admissions that pass. Adding the preference judge reaches 60.4% (+5.8pp), *entirely* from disambiguation (+6 tasks), with hallucination flat—the judge fires exactly where pure-code heuristics were inert (preference reads rose from 18 to 40 trials on disambiguation tasks). It over-fires on base and hallucination, forcing extra preference reads, but at zero Pass³ cost there: both stay flat (base 33→33, hall 32→32 over +69 extra preference reads)—reading a preference cannot make a correct explicit-value action wrong. Base stays

Table 2: Train-split Pass³ by task type. Each cell gives passes (Pass³ rate, %); column headers are the task count.

Config	Base/50	Hall./48	Disamb./31
Baseline, think off	33 (66%)	20 (42%)	11 (35%)
Ours, think off	33 (66%)	32 (67%)	15 (48%)
Ours, think high	36 (72%)	39 (81%)	16 (52%)

Table 3: Test-split Pass³ by task type. Each cell gives passes (Pass³ rate, %); column headers are the task count.

Config	Base/50	Hall./50	Disamb./25
Baseline, think off	32 (64%)	11 (22%)	11 (44%)
Ours, think off	31 (62%)	25 (50%)	13 (52%)
Ours, think high	37 (74%)	30 (60%)	13 (52%)

flat under thinking-off by design—the gates target fabrication, guessing, and policy failures, not the missing-pre-check failures that dominate base; base moves only under reasoning (think-high, +12pp test).

Thinking effect. Turning reasoning on (same V4-Flash model, high effort) adds a further +7.9pp train and +7.3pp test. Reasoning lifts hallucination most (train 67%→81%), and on test it lifts base (+12pp) and hall (+10pp) while disambiguation stays flat (52%→52%): a stronger reasoner helps where the model must reason about state and gaps, not where it must classify an underspecified value. The harness mechanisms and model reasoning are independent and additive—the +12.7pp architecture gain is recovered at either thinking setting, and the two levers stack to 68.3% train / 62.0% test.

5 Discussion

Compute. Input is roughly 85k tokens per trial, dominated by the verbatim policy wiki. At the thinking-off setting a 94.7% prompt-cache hit keeps paid input small (~0.9k output/trial, ~\$0.0011/trial at 1.06× agent LLM time; the confirmation judge adds ~82 ms/trial). Enabling high reasoning raises cost to ~\$0.0042/trial (\$1.61 train and \$1.44 test over the full 3-trial splits) and latency to 27.5 s/trial, driven by the extra reasoning tokens.

Prompt caching over domain routing. A natural token saver is to classify each request into a domain and inject only that domain’s tools and policy. We built and abandoned it. First, CAR-bench tasks sometimes start in one domain and surface another mid-conversation. Adapting then—by reclassifying each turn, or by handing the model a tool to load another domain on demand—changes the tool set mid-task and invalidates the cache. Second, at 58 tools the model is not bottlenecked on tool selection (the domain-routed variant we built exposes fewer tools per turn and did not outperform the full-wiki agent), and a domain-routed prefix would break cross-task reuse: the fixed full-wiki prefix is identical across all tasks and caches at 94.7%, whereas routing gives each domain its own prefix, so tasks in different domains no longer reuse each other’s cache. Injecting the whole policy verbatim

each turn is the cheaper design, and the guardrail absorbs the compliance risk of a long prompt: rules the model glosses over are still enforced at dispatch.

Limitations. (i) The hardest remaining failures are structural: in the navigation-edit hallucination family, when an edit tool is removed the agent can still reach the correct end-state by deleting the navigation and setting a new one, yet the task fails because the rubric scores admission of the missing capability, not a silent workaround. The compliant fix is a policy rule—delete only on an explicit user request—which the benchmark’s own navigation policy does not state (its clauses gate on system state, not user intent); we have not landed it (ceiling +5 tasks). (ii) Some tasks turn on under-specified instructions. In one, the driver heads to Mannheim if it is not raining, else Cologne; the instruction does not say whether to check the weather now or at the estimated arrival time, and the reference checks a later time (plausibly the arrival forecast) while our agent checks the current time. Both readings are defensible but yield different destinations, so our agent scores 0 on the state match. Such tasks blur the ceiling without indicating a clear agent defect.

Future work. Three experiments are out of scope here. (i) A frontier-model study: run a frontier model with and without the harness to test whether the gain substitutes for model scale or diminishes there (we test only one non-frontier model). (ii) A per-gate ablation: ours is cumulative, so each gate’s marginal contribution is not isolated. (iii) A base-targeted gate—e.g., a pre-check checklist gate—for the base parity the present gates do not address.

A Policy clauses enforced by the deterministic gates

Our deterministic gates enforce the benchmark’s numbered policy clauses; the main text describes their effect. The cascade rules **AUT-POL:008–014** are the light and climate rules that require side effects: closing any window open more than 20% and raising the fan when the air conditioning turns on; fog lights must also switch the low beams on and the high beams off; and a high beam must not be lit while the fog lights are on. The confirmation rules **LLM-POL:004, 007, 008** mark three actions as requiring explicit user agreement before dispatch, resolved by the confirmation judge described in the main text.

References

- [Kirmayr *et al.*, 2026] Johannes Kirmayr, Lukas Stappen, and Elisabeth André. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty. *arXiv preprint arXiv:2601.22027*, 2026.
- [Xu *et al.*, 2026] Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, et al. DeepSeek-V4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*, 2026.
- [Yao *et al.*, 2022] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao.

ReAct: Synergizing reasoning and acting in language models. In *NeurIPS 2022 Foundation Models for Decision Making Workshop*, 2022.